Society Chef (HomeChef)

Comprehensive Project Specification, System Agents, Workflows & Tech Stack Document

Document Version: 1.0.0

Target Environment: Production / Monorepo

Scope: Mobile App, Backend API, Ops Portal & Delivery App

Primary Stack: React Native, Expo 54, Express 5, Supabase

1. Executive Summary & Project Purpose

Society Chef (HomeChef) is a hyperlocal, society-first marketplace engineered specifically for residential complexes. It empowers home cooks—primarily homemakers—to launch branded, cloud-style home kitchens inside their apartment flats and supply fresh, hygienic, home-cooked meals directly to fellow society residents and surrounding neighborhoods.

Key Market Problems Addressed:

- **High Commission Barriers:** Traditional food aggregators charge 25-35% margins and require commercial FSSAI licenses that deter home cooks.
- **Trust & Hygiene Gap:** Hyperlocal society visibility fosters trust through resident peer reviews, hygiene compliance verification, and transparent kitchen profiles.
- **Pre-Order Schedules & Waste Reduction:** Integrated Weekly Planners allow chefs to set strict meal prep slots, minimizing food waste.
- **Hyperlocal Logistics:** Direct same-society delivery eliminates high last-mile courier expenses and guarantees piping hot food.

2. System Agents & User Roles

The platform coordinates interactions across four distinct system actors, each equipped with tailored permissions and user interfaces:

System Agent / Role	Target Application Interface	Key Responsibilities & Capabilities
Consumer (Resident)	`apps/mobile` `app/(consumer)`	• Discovers local & same-society kitchens. • Applies automatic 10% same-society discount. • Pre-orders meals & claims weekly free dish vouchers. • Live order tracking & real-time chat with chefs. • Participates in 5-order reward challenge spin wheels.
Home Chef (Cook)	`apps/mobile` `app/(chef)`	• Configures kitchen profile & toggles privacy (`hideIdentity`). • Sets dish pricing, lead preparation time, custom dishes. • Manages day-to-day schedules via Weekly Planner. • Processes incoming order stages (accept, prep, ready). • Views gross sales, payouts, and orders packaging supplies.
Delivery Rider	`apps/delivery`	• Receives order dispatch assignments. • Broadcasts real-time GPS location via WebSockets. • Updates pickup & final delivery milestones. • Handles door-to-door delivery within society towers.
Operations & Admin	`apps/ops-web` `apps/mobile/app/(ops)`	• Inspects hygiene & FSSAI/bank verification requests. • Approves or rejects onboarding kitchen applications. • Monitors active order metrics, platform commission analytics. • Manages packaging supply inventory catalog.

3. Technical Stack & Architecture ('Textile')

Society Chef is engineered as a modern TypeScript monorepo combining mobile, web, backend REST services, and real-time database infrastructure:

Layer / Subsystem	Core Technologies	Purpose & Key Libraries
Mobile Application (`apps/mobile`)	React Native, Expo SDK 54, TypeScript, Expo Router	Cross-platform iOS/Android app. Uses Zustand for client stores, TanStack React Query for async caching, `@react-native-picker/picker`, reanimated.
Delivery App (`apps/delivery`)	React Native, Expo SDK 54, Expo Location, TaskManager	Rider app utilizing background location tasks (`expo-location`) for live courier tracking.
Operations Portal (`apps/ops-web`)	React 18, Vite 5, Tailwind CSS v4, Zustand, Lucide	Admin dashboard for platform operations. Instrumented with Sentry React monitoring, Cypress E2E, and Playwright tests.
Backend API Server (`apps/backend`)	Node.js, Express 5, TypeScript, Zod, Sentry	REST services for payment verification, rate limiting, sharp image processing, Sentry profiling, and push notifications (`expo-server-sdk`).
Database & BaaS (`supabase/`)	Supabase, PostgreSQL, GoTrue Auth, Realtime WS	Hosted Postgres DB with RLS policies, GoTrue Phone OTP auth, storage buckets for dish/kitchen media, and WebSocket channels for chat & live orders.
Payment Gateway	Razorpay SDK, Webhooks	Supports Indian Cards, UPI & Wallets. Server-side webhook signature verification. Automatic platform commission split (10%) and payment fee (2%).

4. End-to-End Application & Data Workflows

The system operates through synchronized workflows across authentication, scheduling, ordering, and settlement:

A. Authentication & Profile Onboarding

1. User enters mobile number in `app/(auth)/phone.tsx`.

2. Supabase GoTrue dispatches SMS OTP, verified in `app/(auth)/otp.tsx`.

3. Role assigned in `role.tsx` (Consumer or Chef).

4. Chef completes onboarding: Kitchen Name, Society Selection, Privacy Toggle (`hideIdentity`), FSSAI document upload, and Bank Account credentials for settlements.

B. Chef Weekly Planner & Availability Flow

1. Chef opens `app/(chef)/menu.tsx` and accesses the Weekly Availability Planner.

2. Selects active dishes, configures quantity limits, and sets close time via alarm-style picker (`CloseTimePicker.tsx`).

3. Nominates one Weekly Free Item or Free Sample dish per week.

4. Save action fires optimistic updates to Zustand `useWeeklyPlannerStore` and persists to Supabase `weekly_dishes` table.

C. Discovery, Pre-Order & Same-Society Discount Flow

1. Consumer opens `discover.tsx`; system filters kitchens by society proximity.

2. If consumer belongs to the same society as chef, a 10% flat discount is automatically calculated during checkout.

3. Consumer adds dishes to cart (`useCartStore`) and opens `CheckoutModal.tsx`.

D. Secure Razorpay Payment & Atomic Stock Reservation

1. Backend `apps/backend` creates a Razorpay Order ID.

2. Frontend launches Razorpay Native Sheet. Upon payment callback, payload is submitted to Express API.

3. Express backend verifies cryptographic Razorpay signature via webhooks/endpoints.

4. Backend calls Supabase Postgres RPC function `decrement_dish_quantity` to atomically reserve remaining portions and prevent overselling.

E. Real-time Order Fulfillment & Live Delivery Tracking

1. Order state transitions: `placed` -> `accepted` -> `preparing` -> `ready` -> `out_for_delivery` -> `delivered`.

2. Chef console (`app/(chef)/orders.tsx`) updates milestone states.

3. Rider app (`apps/delivery`) receives dispatch assignment and streams GPS coordinates.

4. Consumer live tracking screen (`app/(consumer)/track/[order_id].tsx`) renders milestone progress and map location via Supabase Realtime.

F. Financial Settlements & Growth Gamification

1. Upon order completion, backend updates financial ledger `settlements` table: Gross Amount minus 10% platform fee and 2% payment gateway cost.

2. Every 5 completed consumer orders increments the Weekly Food Challenge counter, triggering a wheel-of-fortune spin for discount vouchers.

3. Chefs can purchase standard packaging boxes/containers from the integrated Supplies Shop (`app/(chef)/supplies.tsx`).

5. Monorepo Architecture Overview

Directory Path	Module Role & Description
`apps/mobile/`	Primary Expo React Native application containing Consumer, Chef, and Operations mobile routes.
`apps/backend/`	Node.js Express 5 API for Razorpay webhooks, push notifications, and image optimization.
`apps/ops-web/`	Vite + React admin web dashboard for operational analytics and kitchen trust verification.
`apps/delivery/`	Expo React Native mobile app for delivery riders handling hyperlocal order distribution.
`supabase/`	Database migrations, Postgres schema (`schema.sql`), atomic RPC functions, and RLS policies.
`docs/`	Product requirements (`requirements.md`) and architectural guidelines (`tech-stack.md`).